FUN GC ／ UNITY講座シリーズ

【Unity基礎】

オブジェクト指向をC#で学ぼう

第4回　ポリモーフィズムとインターフェース



2026年10月28日　講師：嘉屋悠月

TODAY ・ 今日のゴール

3つできるようになろう



まとめて持てる

Enemy 型の変数ひとつで、Slime も Boss も扱える



同じ呼び方

同じ Attack() で、種類ごとに違う動きをさせられる



1行で動かす

敵が増えても「1行」で全員に攻撃させられる

POINT　用語は後から。まず動かして体験してから、名前を付けよう。

TODAY・90分

今日の流れ

01	ポリモーフィズムとは	25分
02	virtual と override、List でまとめて動かす	10分
03	インターフェース（interface）	20分
04	演習：Enemy をまとめて動かす	30分
05	まとめ・次回予告	5分

前回、こんな書き方をしたよね

第3回のコードから

```
public class Enemy
{
    public virtual void Attack() { … }
}

public class Boss : Enemy
{
    public override void Attack() { … }
}
```

子が親の Attack を上書きできた。

でも呼ぶときは、

slime.Attack()、boss.Attack()……と

1体ずつ別々に書いていた。

本当にこのままで大丈夫？

SECTION

01

ポリモーフィズムとは

同じ呼び方なのに、動きが変わるしくみを知ろう



同じ呼び方なのに、結果が違う

GameManager.cs ／ Start() の中

```
Enemy a = new Slime();
Enemy b = new Boss();

a.Attack();
b.Attack();
```

a.Attack() の結果

ぺちぺち

b.Attack() の結果

攻撃

ファイア！

? **同じ Enemy 型の変数で、同じ Attack() と呼んだのに、結果が違う。どうして？**

同じ呼び方で、中身の違う動き

多態性（ポリモーフィズム）

「同じメソッド名」を呼んでも、
実際のオブジェクトによって動きが変わる。

呼ぶ側は気にしなくていい

Enemy 型の変数で呼ぶだけ。
中身が Slime か Boss かは、呼ぶ側が知らなくてよい。

前提は「継承」と「上書き」

親に virtual、子に override があると、
子の動きが使われる。

Enemy e = new Slime();	
e.Attack()	→ 呼ばれるのは Slime の Attack
e.Move()	→ 継承のまま（書いていない）
型は Enemy、中身は Slime	

SECTION

02

virtual と override、List でまとめて動かす



第3回の復習から、敵をまとめて動かす書き方へ

親に virtual、子に override

Enemy.cs ／ Slime.cs

```
public class Enemy
{
    public virtual void Attack() { Debug.Log("攻撃"); }
}

public class Slime : Enemy
{
    public override void Attack() { Debug.Log("ぺちぺち"); }
}
```

① 親は virtual で「上書き可」

virtual が付いたメソッドだけ、
子が差し替えられる。

② 子は override で置き換え

同じ名前・同じ引数で書くと、
子の Attack が使われる。

③ 書き忘れると別物になる

override を付けないと、
Slime の Attack は「別のメソッド」扱い。

親の型の List にまとめると便利

GameManager.cs

```
List<Enemy> enemies = new List<Enemy>();
enemies.Add(new Slime());
enemies.Add(new Boss());

foreach (Enemy e in enemies)
    e.Attack();
```

① 型は親の Enemy

Slime も Boss も Enemy の仲間なので、同じ List に入る。

② ループは1回だけ書く

敵が増えても Attack の呼び出しは同じ1行。出力は「ぺちぺち」「ファイア！」と変わる。

SECTION

03

インターフェース

「何ができるか」だけを決める約束と、継承との使い分け



「ダメージを受けられる」という約束

IDamageable.cs

```
public interface IDamageable
{
    void TakeDamage(int amount);
}
```

① 中身は書かない

処理は書かず、名前と引数だけを決める。

② 名前は「I」で始める

慣習として、interface の名前は I から始める。

③ new はできない

interface 自体は new で作れない。守るのは Enemy などのクラス。

継承とインターフェースの使い分け

継承（: 親）

問い	「〜の一種か？」
持てる数	親は1つだけ
中身	共通の変数・処理を持てる
例	Slime は Enemy の一種

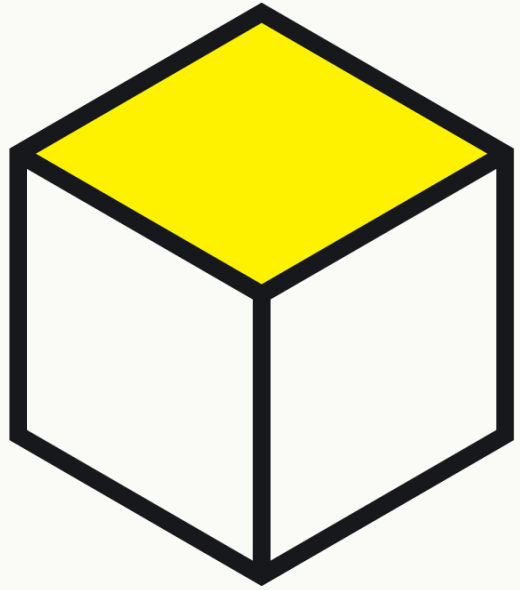
インターフェース（: I〜）

問い	「〜ができるか？」
持てる数	いくつでも実装できる
中身	約束だけ（中身は子が書く）
例	Slime は IDamageable を守る

迷ったら：まず継承を考え、「別の種類の約束にも当てはまる」ときにインターフェースを足す。

POINT ここだけは覚える

親の型で呼べば、
中身の子の動きが動く。



呼ぶ側は、中身が Slime か Boss かを知らなくていい。

04 ・ つまずきポイント

エラーの出やすい3か所

01

override を忘れる

子の Attack に override がないと、警告（CS0114）が出て親の Attack が隠れるだけになる。

02

親に virtual がない

親側に virtual を付けていないと、子で override できずエラーになる。

03

interface の実装し忘れ

約束したメソッドを書かないと、エラー（CS0535）になる。名前と引数は約束どおりに。

EXERCISE

演習

目安時間

30分

Enemy をまとめて動かそう

ゴール　List<Enemy> に入れて、一度に Attack を呼ぶ

1. Enemy.cs：virtual な Attack() を書き、IDamageable を実装する
2. Slime.cs：Attack() を override して「ぺちぺち」と出す
3. Boss.cs：base.Attack() を呼び、「ファイア！」を足す
4. List<Enemy> に Slime と Boss を入れ、foreach で Attack を呼ぶ

早く終わった人へ

Boss に TakeDamage を実装して hp を減らす ／ override を外して CS0114 の警告を見る

05 ・ まとめ

今日できるようになったこと



ポリモーフィズム

親の型で呼んでも、
中身の子の動きが使われる



virtual と override

親は virtual、子は override。
base で親の処理も呼べる



インターフェース

「できること」の約束。
複数つけられる

NEXT

第5回　[次回のテーマ]

[次回予告のひとこと]